

El catálogo completo de AlgoProject — para diseñar la interfaz

Qué es esto. El inventario de todo lo que esta herramienta sabe hacer hoy, organizado por el orden en que se usa, escrito para un agente que va a ayudar a diseñar la aplicación que englobe StrategyQuant X y este proyecto. Las cosas **ya construidas** están en las secciones 1 a 6. Las **que aún no existen** están separadas en la sección 8, y no se mezclan con las anteriores a propósito: una interfaz que reserve sitio para lo que no existe envejece mejor que una que lo descubra tarde, pero sólo si no se confunden.

Generado el 2026-09-25 leyendo el código, no de memoria. Cada comando aquí es copiable; cada número medido dice sobre qué se midió.

Cómo leerlo. La sección 1 es la secuencia: es la columna vertebral y casi con certeza la estructura de la interfaz. La 2 es lo que ya hay de aplicación —hay más de lo que parece—. La 3 es el catálogo grande, por pasos. La 4 dice qué forma tiene cada resultado, que es lo que decide qué hay que pintar. La 5 y la 6 son las reglas que condicionan cualquier diseño. La 7 son los números de coste reales.

1. La secuencia — 21 pasos

Esto lo dictó el dueño y manda sobre cualquier otro orden. Un activo entra por arriba y sale, o no, por abajo.

#	paso	quién lo ejecuta	qué decide
1	La idea, en conversación	persona	—
2	Vocabulario: ¿existe el bloque? Si no, se crea y se instala	agente + SQX	qué se puede expresar
3	Plantilla: bloque fijo + hueco aleatorio, con registro de lo ya probado	agente	qué se va a construir
4	Preflight: costes, ventanas y rangos — bloqueante	Python	si se puede empezar
5	Creación del custom project	Python → SQX	el arnés del estudio
6	Configuración del build	Python → SQX	qué ve el generador
7	Retest OOS en SQX	SQX	la primera muestra fuera
8	Criba IS/OOS en Python → aplicar el veredicto	Python → SQX	de miles a decenas
9	Retest cross-market en SQX	SQX	—
10	Análisis cross-market	Python	¿transfiere o sólo estaba largo?
10.5	Preparación cross-timeframe: hermanas reescaladas	Python	—
11	Retest cross-timeframe en SQX	SQX	—
12	Análisis cross-timeframe	Python	¿sobrevive a un reloj más lento?
13	MC Retest en SQX: 8 perturbaciones	SQX	—
14	Análisis MC Retest	Python	¿qué entrada la rompe?
15	SPPs en SQX: la rejilla de permutaciones	SQX	—
16	Reconocimiento SPP	Python	qué parámetros mandan, y si merece 5.000 variantes
16.5	Fábrica de variantes + retest del lote	Python → SQX	el panel del 17 y el 18
17	Walk Forward Correlation	Python	¿optimizar predice algo?
18	CSCV / PBO	Python	¿mi forma de elegir sobreajusta?
19	Walk Forward Matrix en SQX	SQX	¿reoptimizar selecciona lo que fallará?
20	Lectura conjunta de 17, 18 y 19 — CIEGA hasta tener los tres	—	□ no existe
21	Exposición contra buy & hold	Python	qué tiempo de mercado costó lo ganado

Del 22 en adelante empieza la cartera, que está casi sin empezar.

Tres reglas de la secuencia que la interfaz tiene que respetar o hacer daño:

- **oos2 es una puerta de un solo sentido.** La historia está partida en `build` (2008-2017), `oos1` (2018-2022) y `oos2` (2023-hoy). Los pasos 7 a 16 miran `oos1` una y otra vez; `oos2` está **reservado** para los pasos 17 y 19 y cada mirada lo gasta. Ya no es un acuerdo: el módulo `ledger/` se niega.
- **El paso 20 va ciego.** No se miran los resultados de 17, 18 y 19 hasta que los tres estén hechos, porque ver dos contamina la decisión de correr el tercero. También está forzado.
- **Diagnóstico, nunca selección.** Ningún módulo sustituye una estrategia por un clon que puntúe mejor. Mover un parámetro es decisión de una persona, se anota y se revalida.

2. Lo que ya existe de aplicación

Más de lo que parece, y con la arquitectura ya decidida.

```
bin/algoui → ui/desktop (PySide6) —HTTP loopback→ ui/daemon (FastAPI) → core/, sqx/
```

Dos procesos a propósito. El demonio sobrevive a la ventana: se cierra la ventana, el trabajo sigue. La regla dura: **ninguna vista abre un fichero ni un CSV** — todo pasa por el demonio, que es el único que lee y escribe. El demonio escucha sólo en loopback.

La ventana nunca corre SQX y nunca autoriza nada. El chat escribe un borrador de brief y entrega el prompt; la autoría es una skill que instala bloques en un install real. Una ventana que escribiera en un worker rompería el reparto de carriles en cuanto dos sesiones la tuvieran abierta.

Zonas construidas

zona	qué hace
Cobertura	la matriz de la librería de plantillas: qué se ha probado en qué mercado
Plantillas	la ficha de cada plantilla, con sus corridas y su veredicto
Nueva plantilla	un chat que entrevista una idea hasta dejar el brief escrito y el comando listo
Paletas	la librería de paletas de bloques construida sobre la taxonomía
Activos	los 19 instrumentos y los 4 ficheros compartidos: costes, tramos, rangos y doctrina, editables sin abrir un YAML, conservando los comentarios
Estrategias	los databanks como los agrupa SQX y, por estrategia, qué dijo ya cada módulo de análisis sobre ella

La zona de Estrategias ya conoce **doce módulos por estrategia** y el paso del workflow al que sirve cada uno, y sabe lanzar diez de ellos; de los otros dos explica por qué no puede (uno es una skill, el otro se corre sobre un lote de variantes).

Zonas anunciadas, con página que dice qué irá y qué se hace hoy

zona	pasos	qué se hace hoy en su lugar
Datos	antes del 1	leer a mano ~/Desktop/AlgoData/INDEX.md
Generación	4 a 7	abrir SQX y navegar sus menús, más comandos sueltos y bin/sqx-worker.sh a mano
Estudios	8, 10, 12, 14, 16, 17, 18	tres serve.py distintos en el navegador, cada uno con su puerto, sin estado compartido
Carteras	después del 20	nada: portfolio/ está vacío

⚠ **Los tres paneles de navegador son deuda declarada.** strategies/crossmarket/explorer/, strategies/monteCarlo/explorer/ y strategies/retest/explorer/ levantan cada uno un Flask con su puerto. La decisión del dueño es que **las vistas nuevas van dentro de ui/**, misma app y mismo demonio, y que esos tres se absorban. Que dejen de ser cuatro aplicaciones es la razón por la que el demonio existe.

3. El catálogo, paso por paso

Cada ficha dice: **qué responde**, el **comando** exacto, **qué necesita y qué deja**, y si toca SQX. La columna «UI» dice cómo se hace hoy desde la aplicación.

3.1 · Preparar el estudio — pasos 2 a 6

herramienta	qué responde	entrada → salida	SQX	UI
<code>python3 -m sqx.inspect.vocabulary <término></code>	¿Qué puede expresar este install? Sus bloques, sus grupos aleatorios y qué contiene cada uno	install → listado, -- diff, -- snapshot	lee un install parado	—
<code>/strategy-template (skill)</code>	De una idea a una plantilla: entiende la lógica, mira si el bloque existe, lo autoriza si no, y emite el <code>.sqx</code> en la librería	idea → <code>.sqx</code> + registro	instala bloques en el conductor	zona Nueva plantilla escribe el brief
<code>python3 -m sqx.blocks.install <bloques></code>	Instalar bloques autorizados	XML → install parado	exige SQX parado	—
<code>python3 -m sqx.blocks.taxonomy . .palette</code>	Los 767 bloques que el builder puede sortear de verdad, y las paletas encima	catálogo → YAML	no	zona Paletas
<code>python3 -m sqx.templates.build <nombre> ...</code>	Escribir una plantilla fijando un bloque en un esqueleto probado	bloques → <code>.sqx</code>	no	—
<code>python3 -m sqx.templates.registry</code>	Qué plantillas hay y en qué mercados se han probado	— → <code>registry.csv</code>	no	zonas Cobertura y Plantillas
<code>python3 -m core.assets <SÍMBOLO></code>	Preflight bloqueante: costes, ventanas y rangos del activo, leídos en voz alta	<code>assets/</code> → informe, sale ≠0 si falta algo	no	zona Activos los edita
<code>python3 -m sqx.projects.builder <nombre> --template ... -- symbol ...</code>	Crear el custom project clonado del donante congelado, con sus tareas	plantilla + activo → proyecto instalado	escribe en un worker	—
<code>python3 -m sqx.projects.configure <cfx> <símbolo></code>	Escribir los costes declarados y la ventana de un tramo en cada tarea	activo → <code>project.cfx</code>	escribe	—
<code>/template-run (skill)</code>	Probar una plantilla en un mercado de punta a punta, y comprobar que las estrategias llevan de verdad el bloque fijo	plantilla + símbolo → build + registro	quema CPU	—

Los cinco escritores de cross-checks son el mismo patrón —encender un chequeo en una tarea y escribirle su configuración desde `assets/` — y se usan en sus pasos respectivos: `sqx.projects.crossmarket` (9), `crosstf` (11), `mcretest` (13), `spp` (15), `wfc` (16.5) y `wfm` (19).

3.2 · Cribar la población — pasos 7 y 8

De miles de estrategias a decenas. Es el paso donde más datos se mueven.

herramienta	qué responde	entrada → salida	SQX	UI
<code>python3 -m sqx.export.export_metrics --project P --databank D</code>	Las 41 métricas por estrategia, con columnas IS/OOS emparejadas	databank → <code>metrics.csv</code> (único, se sobrescribe)	lee	—
<code>python3 -m sqx.export.export_trades --project P --databank D --symbol S</code>	Cada operación: horas, precios, tamaño, P&L, MAE/MFE , muestra y motivo de salida	databank → <code>trades.parquet</code> (fechado, inmutable)	lee	—
<code>python3 -m gate.harvest --project P --databank build -oos-databank oos1</code>	La cosecha : une los dos databanks por identidad y se lleva métricas, operaciones y equity diaria de una pasada	dos databanks → 4 ficheros + manifiesto	lee	—
<code>python3 -m gate.report --project P --databank D --feed F</code>	La puerta : ocho cribas en cascada, cada una sobre lo que dejó la anterior	cosecha → <code>scorecard.parquet</code> , <code>funnel.csv</code> , dos <code>verdict.csv</code> , <code>resumen.md</code>	no	lanzable desde Estrategias
<code>python3 -m sqx.curate.verdict ... / apply_verdict ...</code>	Escribir un veredicto a mano, y aplicarlo: borrar del databank lo rechazado, con parada del install y registro de lo que había	veredicto → databank curado	escribe, con -- apply	es una skill: <code>/curate</code>
<code>python3 -m tasks.reports.is_oos · decay · filters · compare · nulls</code>	Las cinco lecturas de población: el panel IS/OOS, el decaimiento por estrategia, el barrido de filtros candidatos, si una conclusión se replica en otra muestra, y cuántas baten a su mono	<code>metrics.csv</code> → informes HTML/MD	no	—

Las ocho cribas de la puerta, en orden (las dos últimas informan y no eliminan):

presencia (si SQX la tiró, fuera) → sanidad (mínimo de operaciones) → estáticas (suelo de métricas OOS) → degradación (cuánto filo sobrevivió y si bate a su error estándar) → forma (si el peor descenso cabe en lo esperado) → mono (si bate a un aleatorio con la misma oportunidad) → familia (corrección por multiplicidad, **soft**) → redundancia (si son N estrategias o una repetida N veces, **soft**).

□ Medido sobre un estudio real: **120 entraron, 45 salieron**. Las estáticas se llevaron casi la mitad (112 → 64) y la degradación otro 30 %.

3.3 · ¿Transfiere el filo? — pasos 9 a 12

herramienta	qué responde	entrada → salida	SQX	UI
<code>python3 -m sqx.projects.crossmarket <símbolo> --cfx ...</code>	Escribe el chequeo de mercados adicionales: cuáles, en qué ventana y a qué costes, desde <code>assets/_markets.yaml</code>	activo → tarea configurada	escribe	—
<code>python3 -m sqx.export.export_retest -- project P --databank D</code>	El retest cross-market a un Parquet, cada mercado dentro	databank → <code>trades.parquet</code> con columna <code>Symbol</code>	lee	—
<code>python3 -m strategies.crossmarket.report ...</code>	¿El filo transfiere, o sólo estaba largo? Cinco modelos nulos de colocación por mercado, reconciliando el fill de cada uno	export → panel por mercado	no	□ panel Flask propio
<code>python3 -m sqx.variants.scale --mothers ... --targets H4</code>	Hermanas con los parámetros en barras reescalados a otro timeframe	<code>.sqx</code> → hermanas + <code>scaling.parquet</code>	no	—
<code>python3 -m sqx.projects.crosstf <símbolo> --cfx ... --task ...</code>	El chequeo cross-timeframe: mismo activo y costes, leído en otros relojes	activo → tarea	escribe	—
<code>python3 -m strategies.crossTF.report -- export ... --scaling ...</code>	¿Sobrevive a un reloj más lento? Con celda de control : la hermana corrida en su propio timeframe, que separa «murió en H4» de «murió al cambiarle los periodos»	export → tabla de celdas con veredicto	no	—

3.4 · ¿Qué la rompe? — pasos 13 a 16

herramienta	qué responde	entrada → salida	SQX	UI
<code>python3 -m sqx.projects.mcretest <símbolo> --cfx ... --input ...</code>	Escribe las ocho tareas de MC Retest: barra inicial, spread, slippage, distancia mínima, parámetros, salidas, OHLC y todo a la vez	activo → 8 tareas	escribe	skill <code>/mcretest</code>

herramienta	qué responde	entrada → salida	SQX	UI
<code>python3 -m strategies.retest.ingest --project P</code>	Lee las ocho, reconcilia cada métrica reconstruida contra SQX y escribe un export fechado e inmutable	8 databanks → parquet	lee	☐ roto hoy, \$8
<code>python3 -m strategies.retest.report --project P</code>	Cuatro preguntas en orden: cuánto duele, cómo duele (un régimen o dos), qué la rompe, y si queda filo pagada la multiplicidad	export → informe + veredictos	no	☐ panel Flask propio
<code>python3 -m sqx.projects.spp <símbolo> --cfx ... --input ...</code>	Las dos tareas SPP: la rejilla de permutaciones, una por ventana, con la aceptación apagada para que sea un mapa y no un filtro	activo → 2 tareas	escribe	skill /spp
<code>python3 -m sqx.export.export_spp --project P --databank D</code>	Los perfiles de permutación a una tabla ancha	databank → parquet	lee	—
<code>python3 -m strategies.sppUltra.report -project P --databank D</code>	Qué parámetros mueven el resultado, cuáles están demostradamente muertos , si la familia entera es ruido, y el diseño de las variantes	rejilla → informe + <code>design_brief.json</code>	no	—

3.5 · ¿Sirve de algo optimizar? — pasos 16.5 a 19

Éste es el tramo más pesado y el que más piezas encadena.

herramienta	qué responde	entrada → salida	SQX	UI
<code>python3 -m sqx.variants.make --brief ... -project P</code>	La fábrica: el diseño en tres estratos (vecindad, factorial grueso saturado, cobertura Sobol) y los cinco controles que existen para fallar	brief → N <code>.sqx</code> + manifiesto	no	—
<code>python3 -m sqx.variants.execute --work <dir></code>	Carga el lote en el custodio y lo retestea	<code>.sqx</code> → <code>retest.csv</code>	trabajo largo	—

herramienta	qué responde	entrada → salida	SQX	UI
<code>python3 -m sqx.variants.collect --work <dir></code>	Métricas por segmento y por unión, unidas al manifiesto (contrato C3)	csv → <code>metrics.parquet</code>	no	—
<code>python3 -m sqx.variants.equity --work <dir></code>	El P&L por día de cada variante, por tramo y por mercado de chequeo	<code>.sqx</code> → <code>equity.parquet</code>	no	—
<code>python3 -m strategies.parameterCloud.report --work <dir></code>	¿El punto elegido es un pico de suerte o una meseta? Quién mueve el resultado (índices de Sobol), si hay superficie que leer, si su forma aguanta año a año, y qué rinde la meseta repartida	lote → cuatro lecturas	no	—
<code>python3 -m strategies.walkForwardCorrelation.report --work <dir></code>	¿Lo que optimiza dentro predice lo de fuera? Con intervalo, y con la palabra «indeciso» cuando no da	panel → <code>wfc.html</code>	no	—
<code>python3 -m strategies.walkForwardCorrelation.pbo --work <dir></code>	¿Mi forma de elegir parámetros sobreajusta? CSCV sobre 924 particiones , una vez por regla de selección	panel → <code>cscv.html</code>	no	—

herramienta	qué responde	entrada → salida	SQX	UI
<code>python3 -m sqx.projects.wfm <símbolo> -- cfx ... --input ...</code>	La Walk Forward Matrix: 30 celdas, y la ventana reservada que sólo esta tarea puede escribir	activo → tarea	escribe	skill /wfm
<code>python3 -m strategies.walkForwardMatrix.report --project P</code>	¿Reoptimizar selecciona lo que va a fallar? Veredicto: predice / ciego / perverso	export → informe	no	lanzable desde Estrategias

3.6 · El veredicto y la forma — pasos 20 y 21

herramienta	qué responde	estado
paso 20	La lectura conjunta y ciega de 17, 18 y 19	☐ no existe — §8
<code>python3 -m strategies.exposure.report -project P --databank D --feed F --symbol S</code>	¿Qué tiempo de mercado costó lo ganado? Ocupación, buy & hold al mismo riesgo con tres convenciones de tamaño, retorno por hora expuesta, y cuánto del movimiento ocurrió estando dentro	☐

3.7 · Lecturas por estrategia que no son un paso

Se pueden correr en cuanto haya operaciones exportadas. Son las que más piden una ficha por estrategia en la interfaz.

herramienta	qué responde	coste
<code>python3 -m nulls.one --project P --databank D --feed F --strategy S</code>	El test del mono: dónde cayó entre miles de versiones imaginarias sobre las mismas velas, y de qué canal viene su filo. Escalera de cuatro peldaños según qué se le entrega al azar	—
<code>python3 -m nulls.verify ...</code>	Las tres comprobaciones que tienen que pasar antes de leer ninguna p	—
<code>python3 -m strategies.monteCarlo.report ...</code>	De qué depende el resultado: del orden, de qué operaciones salieron, de la ejecución o del régimen. Cinco modelos de remuestreo en dos familias, más estrés y régimen	—
<code>python3 -m strategies.profitShape.report --export ... --strategy ...</code>	De qué pocas cosas depende: concentración por operaciones y por meses, independencia de la secuencia, y si la media cambió dentro de la muestra	2,9 s
<code>python3 -m strategies.entryQuality.report --export ... --strategy ...</code>	¿La señal de entrada vale algo por sí sola? Recorrido a favor/en contra contra entradas al azar a las mismas horas, y lo que cuesta llegar tarde	3,3 s

3.8 · Transversales — no pertenecen a ningún paso

herramienta	qué hace
<code>python3 -m ledger.report --study ...</code>	El libro mayor de la búsqueda: una línea por búsqueda durante toda la vida de un estudio, el embudo entero, cuánta historia se ha gastado, y cuántas cosas se han probado en total. Y hace cumplir la puerta del <code>oos2</code> y la ceguera del paso 20
<code>python3 -m ledger.report --check-thresholds</code>	Comprueba que los umbrales escritos siguen siendo los que el código usa. Sale con error si divergen
<code>python3 -m ledger.backfill --gate <dir> ...</code>	Reconstruye el libro hacia atrás desde lo que un informe dejó escrito
<code>python3 -m sqx.export.syncBars</code>	Mantiene la librería de barras M1 al día con SQX. Sólo se guarda el minuto; todo lo demás se calcula
<code>python3 -m sqx.data.update</code>	El «Update all» automatizado, con la GUI cerrada, contando las estrategias antes y después
<code>python3 -m perf.catalogue · perf.disk.report</code>	Qué cuesta cada módulo en tiempo y memoria, y dónde están los bytes del data root
<code>python3 tools/daily_audit.py</code>	La mitad de la auditoría diaria que una máquina puede hacer sola
<code>python3 -m pipeline.run --project P</code>	El encadenador: mete las madres de un databank, vuelve días después y lee los veredictos. Reanudable, con registro de progreso
<code>python3 -m sqx.inspect.dump_project <P></code>	Qué hace de verdad un proyecto de SQX, regenerado a demanda

4. Qué forma tiene cada resultado

Esto es lo que decide qué hay que pintar. Todo lo de arriba produce una de estas siete cosas:

forma	quién la produce	qué necesita la interfaz
Tabla por estrategia	la puerta (<code>scorecard</code>), el decaimiento, los veredictos	orden, filtro, y la identidad además del nombre
Embudo — entraron N, salieron M, criba a criba	la puerta, el ledger	leerse de un vistazo; es la cifra que más se olvida
Curva de equity y conos	Monte Carlo, cross-market, exposición, variantes	muchas curvas a la vez con una real destacada
Distribución con una marca — dónde cayó lo real entre miles de imaginarios	los nulos, el mono, las rachas	histograma y la p al lado
Superficie / rejilla	SPP, nube de parámetros, matriz walk-forward (30 celdas)	mapa de calor con escala discreta y contraste alto
Nube de puntos — dentro contra fuera de muestra	el WFC	un punto por combinación, cuadrantes marcados
Veredicto categórico con su significado	cross-TF (5 lecturas), WFM (predice/ciego/perverso), nube (pico/meseta), forma del beneficio	la etiqueta nunca sola : cada una tiene una frase que la explica, y va con ella

Y un contrato que atraviesa todo y conviene que la interfaz haga visible:

Todo paso de Python que juzgue estrategias emite un CSV con `strategy` y `verdict`, y `/curate` lo aplica sobre el databank para que el siguiente paso de SQX sólo vea supervivientes. Da igual qué lo haya juzgado: el mecanismo no cambia.

⚠ **El nombre no identifica.** SQX renombra al colisionar, y dos databanks del mismo proyecto tienen estrategias **distintas** con el mismo nombre. La identidad es el SHA-256 del XML interno normalizado. Cualquier vista que empareje cosas por nombre acabará mintiendo.

5. Dónde caen los datos

Nada pesado vive en el repositorio. Todo está bajo `~/Desktop/AlgoData` :

carpeta	qué guarda	ciclo de vida
<code>metrics/<proyecto>/<databank>/</code>	el export de métricas	uno solo , se sobrescribe
<code>raw/<proyecto>/<databank>/<fecha>/</code>	operaciones y barras exportadas	fechado e inmutable
<code>harvest/<proyecto>/<databank>/<fecha>/</code>	la cosecha de la puerta: métricas + operaciones + equity de una pasada	fechado e inmutable
<code>reports/<proyecto>/<databank>/<fecha>/</code>	informes y veredictos	se acumulan : el CSV es reproducible, el razonamiento no
<code>pipeline/<proyecto>/<estrategia>/</code>	un lote de variantes entero y su estado	por madre
<code>strategyPermutations/</code> , <code>crosstf/</code>	rejillas SPP y hermanas reescaladas	por estudio
<code>ledger/<estudio>.jsonl</code>	el libro mayor de la búsqueda	sólo se añade , nunca se reescribe
<code>bars/</code> , <code>barsDerived/</code>	la librería M1 y los timeframes calculados	única fuente de velas
<code>logs/</code> , <code>snapshots/</code> , <code>projectsBackup/</code>	registros y el donante congelado	—

6. Las reglas que condicionan cualquier interfaz

1. **Tres installs de SQX, con carriles distintos.** El **maestro** es del dueño y su GUI manda: la aplicación no escribe ahí. El **conductor** es para autoría y consultas; el **custodio**, para el único trabajo largo a la vez. Varias sesiones comparten los workers, así que parar uno mata lo que esté corriendo otro.
2. **Hay cosas que exigen SQX parado:** instalar bloques, cargar una configuración, actualizar datos. La interfaz tiene que saber si un install está vivo antes de ofrecer el botón.
3. **Un trabajo puede durar horas.** SQX no se consulta por progreso: **publica** eventos, y el progreso fino viaja en el nombre del hilo. Esto es exactamente lo que el demonio existe para sostener, y por lo que la ventana puede cerrarse sin matar el trabajo.
4. **Nunca se edita un `project.cfx` que un install tiene abierto:** SQX lo reescribe al salir y el cambio se pierde en silencio.
5. **Cada sync de SQX borra los `.sqx` en disco que no tenga en memoria.** Cualquier acción que reinicie un install puede llevarse databanks enteros.
6. **`oos2` se gasta al mirarlo**, y el paso 20 es ciego hasta tener los tres. No es un aviso: el ledger lanza. La interfaz debería **mostrar cuánto queda virgen**, no sólo impedirlo.
7. **Los umbrales se congelan antes de mirar.** Están en `ledger/thresholds.yaml` con quién los puso y cuándo. Los activos y sus costes **sí** se editan desde la aplicación (y es de las cosas que más se usan); los umbrales de decisión, no a la ligera.
8. **Diagnóstico y puerta no son lo mismo.** La mayoría de estos módulos describen; sólo unos pocos eliminan. Una interfaz que los presente igual invita a convertir un diagnóstico en un filtro nuevo, que es una búsqueda más y hay que anotarla.

7. Lo que cuestan las cosas, medido

operación	coste
cargar los 7.949.285 minutos del oro	0,4 s
cosechar el P&L por día de 962 variantes	1,5 s , 5,9 MB
exportar las operaciones de unos miles de estrategias	~90 minutos
la nube de parámetros sobre 998 variantes × 3.925 días	1,9 s , 410 MB
la forma del beneficio de una estrategia (1.119 operaciones)	2,9 s
la calidad de la entrada (200 sorteos)	3,3 s
fabricar 5.000 variantes	6 s a 65 s según la forma del fichero, 70 a 631 MB
un build o un retest en SQX	de minutos a horas

La máquina: 48 núcleos físicos (96 lógicos, y el SMT **resta** en esta carga), 128 GB de los que el maestro reclama 71 y el custodio 80. Lo que limita no son los núcleos, es el ancho de banda de memoria.

8. LO QUE AÚN NO EXISTE

Todo lo anterior está construido y se puede abrir hoy. Esta sección es lo que no, y va separada para que no se confunda. Cada cosa lleva qué la bloquea y en qué zona de la interfaz caería.

8.1 · Encargos escritos, con el diseño cerrado

Existen como encargo autocontenido en `docs/encargos/` : el diseño está decidido, falta construirlo.

#	qué añadiría	qué lo bloquea	zona
9	La población de monos de punta a punta: 10.000 estrategias sin ningún filo por los 21 pasos, contando cuántas salen vivas. «Si de diez mil monos llegan 4 y de diez mil tuyas llegan 5, ya sabes lo que vale ese 5»	nada técnico; tiene caducidad: hay que hacerlo antes del primer pase completo, no después	Estudios
10	SPA de Hansen y StepM de Romano-Wolf sobre la población superviviente: qué estrategias baten al benchmark descontada toda la búsqueda. Complementa al CSCV, no lo sustituye	nada técnico	Estudios
11	Filo por operación en unidades de coste: el edge en spreads, el coste de breakeven, la curva de expectativa contra multiplicador de coste, desglosado por hora de entrada	la comisión porcentual puede cobrar por pata o por operación — un factor de 2 sin medir	Estrategias
12	Tests estructurales: ablación de reglas (quitar una condición y medir), inversión de la señal, y un bloque de entrada aleatoria sembrado dentro de SQX. La ruta XML ya está investigada	hay que editar la <i>lógica</i> del <code>.sqx</code> , y la población actual no tiene stops contra los que validar	Generación
14	Mapa de rendimiento condicional: cada operación clasificada por régimen de volatilidad, tendencia, sesión y día	nada técnico; va el último a propósito , es el único que fabrica hipótesis en vez de comprobarlas	Estrategias
15	Una superficie de parámetros por mercado , y si la región buena coincide entre mercados	faltan los costes acordados de 16 activos, y son $J \times$ mercados backtests	Estudios
16	El simulador de replay: reejecutar cada operación desde una entrada desplazada, recalculando stops y tamaño. La versión honesta del test de retraso	el criterio de aceptación necesita una población con stops, y ésta no tiene ninguno	Estrategias
17	Calidad del feed y atribución de ticks malos: picos, precios estancados, huecos, y qué parte del beneficio los toca	los umbrales son del dueño y se fijan de antemano	Datos
6	Etiquetar los 767 bloques por familia	nadie lo ha corrido	Paletas
13	Descomposición alfa/beta	aparcado por el dueño: primero se cierra la secuencia individual, y el factor que más valdría —los retornos de las EAs que ya se operan— todavía no existe	Carteras

8.2 · Huecos que no tienen encargo

qué falta	por qué importa	zona
El paso 20	No hay nada que lea junto lo que dijeron el WFC, el CSCV y la matriz. Es el paso que cierra la secuencia individual, y no existe	Estudios
Las carteras	<code>portfolio/</code> está vacío. Lo acordado: la correlación se mide entre curvas de equity y no entre métricas; el drawdown se calcula sobre la curva agregada, nunca sumando; las reglas de una cuenta fondeada son restricciones, no filtros a posteriori	Carteras
La migración de umbrales	<code>ledger/thresholds.yaml</code> es hoy un registro : cada módulo sigue leyendo su número de su propio <code>config.yaml</code> , y un comando comprueba que no divergen. Falta que lo lean de ahí	transversal
Cinco pasos sin skill	10, 14, 16, 17 y 18 no se pueden lanzar sin intervención humana, así que la cadena no corre entera sola	Estudios
Los costes de 16 de 17 activos	Sin ellos el paso 9 se niega a escribirse, y cualquier diseño multimercado está parado	Activos
Tres paneles en el navegador	Cross-market, Monte Carlo y MC Retest levantan cada uno su Flask con su puerto y sin estado compartido. Deben absorberse en la aplicación	Estudios
El análisis del MC Retest está roto hoy	Exige las ocho tareas y aborta sin la de distancia mínima, que nunca se escribe en una población a mercado — y todas lo son. Además no sabe leer un worker. El paso 14 no se puede leer hasta arreglarlo	Estudios

9. Lo que hay que decidir, y donde este documento no opina

Para arrancar la conversación de diseño, las preguntas que el catálogo deja abiertas:

1. **¿Cuál es la unidad de navegación?** Hay tres candidatas y las tres aparecen en el código: el **databank** (como agrupa SQX, y como ya lo hace la zona de Estrategias), la **estrategia** (que es donde viven doce de los módulos), y el **estudio** (activo + timeframe + plantilla, que es la unidad a la que el ledger le debe la corrección por multiplicidad). Elegir mal obliga a reescribir vistas.
2. **¿Cómo se representa algo que se gasta al mirarlo?** `oos2` no es un permiso binario: es un recurso que se consume. Hoy se muestra como «leído 0 veces». Merece mejor.
3. **¿Cómo se enseña la diferencia entre describir y eliminar?** La mayoría de módulos describen. Si la interfaz pone un botón igual junto a cada uno, invita a convertir cualquier diagnóstico en un filtro, que es exactamente lo que la metodología prohíbe sin anotarlo.
4. **¿Qué pasa con un trabajo de tres horas?** Cola, progreso, cancelación, y qué ve alguien que abre la ventana a la mitad. El demonio ya está preparado; la vista no existe.
5. **¿Cuánto del paso previo tiene que ver alguien antes de lanzar el siguiente?** La cadena es siete filtros en serie sobre la misma población y cada uno ajusta su umbral mirando lo que mató el anterior. Una interfaz que facilite ese ajuste está facilitando sobreajustar.
6. **¿Se rotula en español o en inglés?** El código, los README y `knowhow/` están en inglés; el manual, los encargos y estos dossiers, en español, porque su lector es el dueño. La interfaz hasta ahora está en español.

Dos apuntes sobre este documento

Está en español porque la interfaz y la conversación que va a acompañar lo están. El otro dossier de esta serie — `capabilities-2026-09-24` , escrito para un agente que propone tests nuevos— está en inglés por la misma razón invertida. Si hace falta en inglés, se traduce.

Es un inventario, así que caduca. Se regenera cuando se construya un módulo o se cumpla un encargo. Un catálogo que subestima lo que existe provoca trabajo duplicado, que es justo lo que viene a evitar.